

DTA-JEPA: Dual-Timescale Recursive Adaptive Latent World Models

with Uncertainty-Allocated Test-Time Adaptation and Persistent Memory

Sheng Cheng^{a,*}

^a Hefei Jiuzhai Big Data Technology Co., Ltd., Hefei, China

* Corresponding author. E-mail: c.sheng@cumt.edu.cn

October 8, 2026

Abstract

Latent world models plan by rolling out a learned predictor in a compact latent space, yet the predictor is almost always frozen after training, and its predictions silently degrade under test-time distribution shift. AdaJEPA showed that a single self-supervised gradient step per model-predictive-control (MPC) replan already recovers much of that loss. We argue that this is the wrong unit of adaptation: a fixed one-step update applies the same compute and the same plasticity whether the world model already fits the test environment or is being asked to predict dynamics it has never seen, and it throws away everything learned at the end of an episode.

We introduce **DTA-JEPA**, a latent world model whose adaptation is *dual-timescale* and *uncertainty-allocated*. The predictor is split into a slow *S-module* that produces a dynamics context from a short history and is never modified online, and a fast *R-module* that recursively refines the next latent through K_t uncertainty-gated refinement steps with deep supervision at every depth. A single surprise signal—the Mahalanobis prediction residual under the model’s own Gaussian—allocates all online resources: gradient steps, learning rate, refinement depth, planning conservatism, memory priority, and rollback. Adaptation is confined to a rank-4 LoRA fast parameter set on the R-module plus the encoder head, protected by an anchor ball and a validation-gated rollback; slow parameters are consolidated only at episode boundaries under an EWC penalty, and fast parameters are stored in a fingerprint-indexed skill bank that warm-starts later episodes.

We replicate AdaJEPA’s evaluation protocol—shape, visual, dynamics and layout shifts, with goals sampled as future frames of held-out trajectories—on a self-contained numpy benchmark, and compare Frozen, a faithful AdaJEPA re-implementation and DTA-JEPA under an identical planner, budget and episode set.

The empirical result is negative and we report it as such: at equal budget the three methods are statistically indistinguishable (combined 43.6, 42.8 and 44.7% over 38 conditions, per-condition spread 21–22 points), and neither the uncertainty allocation nor the meta-learned initialisation produces a measurable gain. We also report the three measurements that explain why, because they constrain the design space more than the headline comparison does: (i) recursive refinement reduces one-step latent error by 42% from depth one to depth three and *degrades* at depth four; (ii) the predicted variance is rank-informative (Spearman 0.55–0.64) but scale-compressed (relative calibration error 82%), so allocation on absolute thresholds saturates and only ratio-based allocation is sound; and (iii) with 594.7k fast parameters the anchor ball of radius 0.8 binds on essentially every gradient step, so the safety layer—not the learning rate—sets the effective step size. A fourth finding is about the benchmark itself: every photometric corruption we apply is a no-op for a latent-distance planner that

sees the corruption consistently in both the current and the goal frame (blur changes the one-step error by 0.02%).

Keywords: latent world model; joint-embedding predictive architecture; test-time adaptation; model predictive control; uncertainty; continual learning; recursive reasoning

1 Introduction

1.1 Background and Significance

Joint-Embedding Predictive Architectures (JEPAs) have become a leading recipe for model-based control from high-dimensional observations: an encoder maps observations to a compact latent space, a predictor models the latent dynamics, and a planner—most often model predictive control (MPC)—optimises a short action sequence by rolling the predictor forward and minimising a latent goal distance [1, 2, 3, 4, 7]. The appeal is that planning never touches pixels, so the model is cheap, and that training needs no reward labels.

In practice the world model is *frozen* after training, and its usefulness at deployment is decided entirely by how well the training distribution covers the test environment. Yet test environments shift: objects change shape, cameras change colour balance, noise appears, masses and frictions change, and room layouts differ. Under such shifts latent rollouts drift, MPC optimises the wrong objective, and success collapses—even for high-capacity models [2, 13, 12].

AdaJEPA [5] made a strong and useful observation: the world model does not have to stay frozen. Inside the MPC loop, the transition that results from every executed action is a free self-supervised label. AdaJEPA plans, executes an action chunk, takes one gradient step on the predictor’s last transformer block and the encoder head using the observed transition, and replans with the updated model. One step per replan already recovers a large fraction of the loss under visual and dynamics shifts, at negligible latency.

1.2 The limitation we address

AdaJEPA’s adaptation is *uniform*: the same one gradient step, at the same learning rate, applied at the same layers, whether the model already predicts the current environment well or is being asked about dynamics it has never seen. Three consequences follow.

1. **Misallocated compute.** In-distribution episodes are updated as hard as out-of-distribution ones. Adaptation is not free: extra gradient steps can *hurt* when the model is already adequate, because they fit the model to a five-transition buffer and move it off the pretrained manifold.
2. **Misallocated plasticity.** Directly updating the pretrained predictor weights offers no structural guarantee that the update is local; the setting where adaptation matters most (unseen dynamics) is also the setting where a large step is most damaging.
3. **No memory across episodes.** AdaJEPA’s evaluation—and its design—reset the model at every episode. Everything learned about an environment is discarded when the episode ends, so nothing accumulates: the model can never get better at a *family* of environments, only at the current one.

A fourth, subtler issue is architectural. A single feed-forward predictor spends the same amount of computation on every prediction. Recent work on reasoning with small networks suggests the opposite is better: improve an answer by recursing on it in latent space, with deep supervision at every recursion depth, and let the number of recursions depend on difficulty [32, 31, 33]. Difficulty-dependent compute is exactly the missing knob for a world model under shift.

1.3 Approach and contributions

We propose **DTA-JEPA**, a dual-timescale recursive adaptive latent world model. Our contributions are:

1. **Computational dual timescale (§3).** The predictor is split into a slow *S-module* that turns a short latent-action history into a dynamics context c_t , and a fast *R-module* that recursively refines the next latent $\hat{y}^{(k)} = \hat{y}^{(k-1)} + g_{\theta_f}([\hat{y}^{(k-1)}, c_t], u_t)$ for $k = 1, \dots, K_t$, with deep supervision over all depths. The slow module is the pretrained anchor and is never modified online; all plasticity lives in a rank-4 LoRA fast parameter set attached to the R-module and the encoder head.
2. **Uncertainty as a resource allocator (§3.4).** A single scalar—the Mahalanobis surprise of the observed transition under the model’s own predicted Gaussian—drives the number of gradient steps U_t , the learning-rate multiplier, the refinement depth K_t , the planning penalty, the memory write priority and the rollback decision. Compute is therefore spent where the model is measurably wrong, and saved where it is right.
3. **Dual memory and safe consolidation (§3.6).** An episodic memory of high-surprise latent transitions (persistent across episodes) plus a fingerprint-indexed bank of fast-parameter snapshots give cross-episode accumulation without catastrophic forgetting; an anchor ball on θ_f , a validation-gated rollback and a latent support penalty in the planning cost bound the damage any single update can do. Slow parameters are consolidated only at episode boundaries under an EWC penalty.
4. **Protocol-level replication and empirical comparison (§4, §5).** We replicate AdaJEPA’s four shift families (shape, visual, dynamics, layout) plus a continual environment-switching suite on a self-contained benchmark, and compare Frozen, a faithful AdaJEPA re-implementation and DTA-JEPA under an identical planner, horizon, replay buffer and episode set, together with five ablations.

1.4 What we do not claim

Our benchmark is not PushT or MuJoCo. It is a physics-based 2D re-implementation built to make the *evaluation protocol* comparable—the same shift families, the same segment-based goal sampling, the same MPC budget, the same success criteria—rather than the same pixels. We therefore report relative comparisons between methods evaluated under identical conditions, not absolute numbers transferable to the original benchmark.

2 Related Work

2.1 JEPA world models and latent planning

LeCun’s proposal for autonomous machine intelligence [1] places a joint-embedding predictive architecture at the centre of a world model: predict in representation space, not in observation space, and use the learned latent dynamics for planning. DINO-WM [2] showed that this works zero-shot on pre-trained visual features, planning with either gradient descent or the cross-entropy method. Sobal et al. [3] argue that reward-free offline trajectories are sufficient for planning with latent dynamics models. Wang et al. [4] add a temporal-straightening criterion that makes latent trajectories nearly linear, which conditions gradient-based planning better; AdaJEPA [5] is built on top of that recipe. LeWorldModel [6] removes the remaining fragility by training a JEPA end-to-end from pixels with only two loss terms—a next-embedding prediction loss and a regulariser that pushes latent embeddings towards an isotropic Gaussian—with no exponential moving average, no pre-trained encoder and no auxiliary supervision; it also uses prediction surprise to detect physically implausible events. V-JEPA 2 [7] demonstrates the same paradigm at foundation-model scale.

We adopt LeWorldModel’s training philosophy, because it makes a from-scratch, single-machine, end-to-end study feasible, and we use its surprise notion as the resource signal of our adaptation controller.

2.2 Test-time adaptation

Test-time training and adaptation (TTT/TTA) update a pre-trained model on the test input using an unsupervised objective: auxiliary rotation prediction [15], entropy minimisation [16], augmentation consistency [17], masked reconstruction [18], and stabilised variants for wild, non-stationary streams [19, 20, 21]. The literature is almost entirely about classifiers, where the adaptation signal is the model’s own confidence. In latent planning the situation is different: the environment hands the agent a labelled transition (o_t, a_t, o_{t+1}) for free, so the adaptation objective is exactly the pretraining objective, and it can be optimised in a closed loop with control.

AdaJEPA [5] is the closest work to ours and our main baseline: it adapts inside the MPC loop, one step per replan, restricted to the predictor’s last transformer block and the encoder head, with a bounded replay buffer of the most recent transitions. In their ablations the adaptation target is chosen by hand per environment family, and stronger adaptation (larger learning rate, more steps, larger buffer) is sometimes beneficial out of distribution—but only when latency permits, and at the cost of stability in distribution. Our work replaces those hand-set knobs with an uncertainty-driven controller and makes adaptation stateful across episodes.

2.3 Adaptation in planning and control

Online model adaptation is classical in adaptive control, from self-adapting IDCOM [25] to online system identification in Dyna-style model-based RL [26, 27]. Modern world-model work adapts predictors with latent action models [22], residual latent-state dynamics [23], or data synthesis during training to close the train–test gap [14]; all of them require either target-domain fine-tuning data, extra online rollouts, or an outer self-improvement loop. TD-MPC2 [24] learns a latent dynamics model and a terminal value function online but optimises a reward objective rather than a self-supervised latent prediction. We stay strictly in the reward-free, self-supervised regime.

2.4 Recursive and hierarchical latent computation

The Hierarchical Reasoning Model [31] couples a slow, high-level recurrent module with a fast, low-level one, and lets the fast module iterate many times per slow update; HRM-Text [33] scales that architecture to a 1B language model and shows that the hierarchical recurrence substitutes for a large amount of pretraining compute. Tiny Recursive Models [32] simplify the idea to its core: a $\sim 7\text{M}$ -parameter network recursively improves a candidate answer in latent space, updating a latent state and then the answer, with supervision applied at every improvement step, reaching strong puzzle-solving performance without scale.

We take the two structural lessons—*two coupled timescales* and *recursion with deep supervision*—and place them inside a world model’s predictor. To our knowledge this is the first recursive-refinement JEPA predictor, and the first use of refinement depth as an adaptation-time compute knob.

2.5 Continual learning and safety of online updates

Catastrophic forgetting [28] is usually mitigated with elastic weight consolidation [29] or replay [30]; in streaming TTA, parameter restoration and sharpness-aware variants are used [20, 21]. Anchoring to a pretrained reference is the standard anti-drift device. Our safety layer is deliberately minimal and auditable: a Euclidean anchor ball on the fast parameters, a validation gate on a held-out slice of the replay buffer with rollback, an immutable slow anchor, and a support penalty expressed as a Mahalanobis radius computed from the training latents.

2.6 Reproducibility-focused benchmarks

PushT and PushObj [10, 2] and PointMaze [11] are the standard test beds for latent planning, and the diverse-maze protocol [12] standardises layout shift. Reproducing these requires the original assets (expert datasets, pre-trained checkpoints and MuJoCo models); where those are unavailable, a physics-based re-implementation that preserves the protocol is a useful alternative. We build ours with numpy only—impulse-based contact physics and a software rasteriser—so that the entire study, from data generation to planning evaluation, runs on a laptop CPU with no external simulator, and so that every shift condition is generated by an explicit, inspectable parameter.

3 Method: DTA-JEPA

3.1 Problem setup

We consider trajectories of observations $o_t \in \mathbb{R}^{n_o}$ (RGB frames plus proprioception), actions $a_t \in \mathbb{R}^{n_a}$, and a goal observation o_g . A latent world model consists of an encoder E_s , an action encoder E_a and a predictor F :

$$z_t = E_s(o_t, p_t), \quad u_t = E_a(a_t), \quad (\hat{y}_{t+1}, \Sigma_{t+1}) = F(z_{t-K+1:t}, u_{t-K+1:t}, u_t), \quad (1)$$

trained on reward-free transitions $\mathcal{D}_{\text{off}} = \{(o_t, a_t, o_{t+1})\}$ by latent prediction. Goal-conditioned planning minimises a latent cost over an action sequence of horizon H , MPC executes the first c actions, and the loop repeats.

The adaptation problem: after each executed chunk we observe a new transition (o_t, a_t, o_{t+1}) and may update a subset Ω of parameters before replanning. AdaJEPA takes exactly one gradient step on $\Omega = \{\text{predictor last block, encoder head}\}$ at a fixed learning rate. DTA-JEPA makes three changes: *what* is updated (a decoupled fast parameter set), *how much* (an uncertainty-allocated budget), and *for how long* (state carried across episodes).

3.2 Encoders and goal conditioning

The state encoder is a from-scratch convolutional trunk followed by an adaptable projection head,

$$z_t = \text{LN}\left(W_h[\text{body}(o_t); \phi(p_t)]\right), \quad \text{body} : 64 \times 64 \times 3 \xrightarrow{4 \text{ conv stages}} 128 \times 4 \times 4, \quad (2)$$

where ϕ is a small MLP over proprioception. The final LayerNorm is not cosmetic: without it, the cheapest way for W_h to reduce prediction error is to shrink every latent towards zero, which is exactly the collapse we observed and had to fix at the source (§7.3). Normalising the output bounds the per-sample scale, so the encoder must encode *direction*.

Two design choices follow the human-inspired architecture of Cerebro [34] and LeWM’s stability argument [6]: (i) no pre-trained semantic encoder and no EMA—we train everything end-to-end from pixels; (ii) fusion happens *after* prediction: the goal is encoded with the same encoder, and the interaction between the current prediction and the goal takes place in the cost function, not by concatenating modalities before the predictor. All modalities share one latent width d .

Proprioception deliberately excludes the manipulated object’s pose: including it would let the model bypass vision entirely and would make the visual-shift suite vacuous. The encoder head W_h and ϕ belong to the fast parameter set; the trunk belongs to the slow set.

3.3 Dual-timescale recursive predictor

Slow module (context). A short history of $K = 3$ latent–action pairs is embedded into K tokens, $x_i = W_s[z_i; u_i]$, and passed through L_S pre-norm transformer blocks; the last token is the dynamics context

$$c_t = \text{Transformer}_{L_S}(x_{1:K})_K. \quad (3)$$

c_t is the slow state: it summarises ”what the dynamics look like here” and is the only channel through which the slow timescale influences prediction. It is deliberately *never* updated online.

Fast module (recursive refinement). The next latent is obtained by recursing in latent space from the context,

$$\hat{y}^{(0)} = c_t + u_t, \quad \hat{y}^{(k)} = \hat{y}^{(k-1)} + g_{\theta_f}([\hat{y}^{(k-1)}, c_t] + u_t), \quad k = 1, \dots, K_t, \quad (4)$$

where g_{θ_f} is a single pre-norm transformer block, and the update is residual. The fast parameter set is not fixed by the architecture: our default folds any trained low-rank adapters into the base weights and adapts the last block directly (which is what our AdaJEPA baseline does, so the two methods differ *only* in the adaptation policy); adapting through rank-4 LoRA instead is reported as an ablation (§5). Each depth k carries its own mean head W_μ and log-variance head $W_{\log \sigma^2}$:

$$\hat{\mu}^{(k)} = W_\mu \text{LN}(g_{\theta_f}^{(k)}), \quad \log \hat{\sigma}^{2(k)} = W_{\log \sigma^2}(\hat{\mu}^{(k)}). \quad (5)$$

The number of refinement steps $K_t \in [K_{\min}, K_{\max}]$ is chosen by the controller (§3.5); the module also reports a per-sample halting depth k^* , the first depth at which the relative change $\|\hat{y}^{(k)} - \hat{y}^{(k-1)}\| / \|\hat{y}^{(k-1)}\|$ falls below ϵ .

This is the HRM/TRM structure transplanted into a world model: a slow module supplies a context (HRM’s high-level module) and a fast module iterates on a latent answer (TRM’s recursive improvement), with supervision at every depth. Two properties matter for adaptation: the refinement is parameter-efficient (the fast set is $\sim 0.3\text{M}$ parameters), and additional *compute* is a first-class axis of adaptation—under shift we can spend more refinement steps instead of larger gradient steps.

Multi-step rollout and inverse dynamics. Planning rolls the same module forward, re-inserting each predicted latent into the history. An auxiliary inverse-dynamics head $G_\omega([z_t; \text{sg}(z_{t+1})]) \rightarrow a_t$ grounds the latent in actions.

3.4 Uncertainty decomposition and surprise

We distinguish three quantities, each with a distinct role.

Aleatoric uncertainty is the predicted diagonal Gaussian: $\mathcal{L}_{\text{unc}} = \frac{1}{2}[(\hat{y} - \text{sg}(z))^\top \Sigma^{-1}(\hat{y} - \text{sg}(z)) + \log \det \Sigma]$.

Epistemic uncertainty is the MC-dropout spread of the R-module, $\Sigma_{\text{epi}} = \text{Var}_m[\hat{\mu}_m]$ over $m = 4$ stochastic passes, computed only when the controller requests it.

Surprise is the Mahalanobis residual of the transition actually observed,

$$s_t = (\hat{y}_{t+1} - \text{sg}(z_{t+1}))^\top (\Sigma_{\text{al}} + \Sigma_{\text{epi}})^{-1} (\hat{y}_{t+1} - \text{sg}(z_{t+1})), \quad (6)$$

i.e. "how implausible is what just happened under the current model". Following LeWM [6], surprise is the natural detector of events the model cannot represent; we promote it from a diagnostic to the control signal of the whole adaptation loop.

3.5 Uncertainty-allocated adaptation controller

Let $\bar{s}_t$ be an exponential moving average of surprise and let $r_t = s_t/\bar{s}_t$ be the relative surprise. Define the allocation

$$\pi_t = \sigma(\gamma(r_t - 1)) \in (0, 1), \quad U_t = \lfloor U_{\min} + \pi_t(U_{\max} - U_{\min}) \rfloor, \quad \eta_t = (1 - \pi_t)\eta_{\min} + \pi_t\eta_{\max}, \quad (7)$$

$$K_t = \lfloor K_{\min} + \pi_t(K_{\max} - K_{\min}) \rfloor. \quad (8)$$

With defaults $[U_{\min}, U_{\max}] = [1, 3]$, $[\eta_{\min}, \eta_{\max}] = [0.2, 5] \times \eta_f$ and $[K_{\min}, K_{\max}] = [1, 4]$, this yields a concrete statement: *when the model already predicts the environment, DTA-JEPA is as cheap as AdaJEPA (one step, short recursion); when the model is wrong, it spends steps, learning rate and refinement depth in proportion to how wrong it is.* The same π_t scales the planning penalties β, γ (§3.7) and the memory write priority (§3.6).

3.6 Dual memory

Episodic memory stores latent transitions (z, u, z') —never pixels—with priority

$$\text{prio}(z, u, z') = \lambda_1 s + \lambda_2 \text{tr } \Sigma_{\text{epi}} + \lambda_3 \text{novelty}(z), \quad \text{novelty}(z) = \exp(-\min_m \|z - z_m\|^2 / \tau). \quad (9)$$

It is a bounded priority queue, so the model keeps the transitions that were hardest to predict rather than merely the most recent ones. Adaptation retrieves the top- k memories by a Gaussian similarity kernel and adds their prediction loss:

$$\mathcal{L}_{\text{on}} = \mathcal{L}_{\text{ada}}(\mathcal{B}_t) + \lambda_{\text{ret}} \mathcal{L}_{\text{ada}}(\mathcal{M}_{\text{ret}}). \quad (10)$$

Parametric memory is a skill bank: fast-parameter snapshots indexed by an environment fingerprint fp (the mean latent of the episode’s first frames). At episode start the nearest snapshot warm-starts θ_f . This is what turns per-episode adaptation into cross-episode learning: returning to a previously seen environment is no longer a cold start.

Parameterisation is an experimental control, not a contribution. Because AdaJEPA’s own ablations show that rank-4 LoRA adapters recover less of the frozen-to-adapted gap than direct updates of the same block, evaluating a LoRA-based method against a direct-update baseline would confound the adaptation *policy* with the adaptation *parameterisation*. We therefore fold the trained adapters into the base weights (numerically exact: the folded and unfolded models agree to 3×10^{-8} in prediction loss) and give both methods the identical fast set—the predictor’s last block plus the encoder head, with the encoder trunk and the S-module frozen—so that measured differences are attributable to allocation, memory, safety and refinement depth alone. The LoRA parameterisation is then a separate row in the ablation table, and it costs accuracy there, which is consistent with AdaJEPA’s finding.

Slow consolidation happens once per episode, on the episodic memory, with an EWC penalty: $\theta_s \leftarrow \theta_s - \eta_s \nabla_{\theta_s} [\mathcal{L}_{\text{ada}}(\mathcal{M}_e) + \lambda_{\text{EWC}} \sum_i F_i(\theta_{s,i} - \theta_{s,i}^*)^2]$ with $\eta_s = 10^{-6}$. The fast parameters are never consolidated into the slow ones during an episode, and the slow ones are never touched by online gradients.

3.7 Safety layer

Four mechanisms bound the damage of a bad update.

1. **Anchor ball.** $\|\theta_f - \theta_f^0\|_2 \leq \epsilon_f$, enforced by projection after every step: $\theta_f \leftarrow \theta_f^0 + \frac{\epsilon_f}{\|\theta_f - \theta_f^0\|}(\theta_f - \theta_f^0)$.
2. **Validation-gated rollback.** The oldest entries of the replay buffer form a held-out slice; if $\mathcal{L}_{\text{val}}^{\text{new}} > \mathcal{L}_{\text{val}}^{\text{old}} + \varepsilon$, the update is reverted and the event is logged.
3. **Immutable slow anchor.** θ_s (encoder trunk and S-module) receives no online gradient: the pre-trained dynamics context cannot be corrupted by a bad episode.
4. **Support penalty in planning.** Let $\tau_{0.95}$ be the 95th percentile of the training latent Mahalanobis radius. The planner minimises

$$C(a) = \sum_{k=1}^H \left[\underbrace{\|\hat{y}_k - z_g\|^2}_{\text{goal}} + \underbrace{\beta \text{tr} \Sigma_k}_{\text{uncertainty}} + \underbrace{\gamma \max(0, \text{Maha}(\hat{y}_k) - \tau_{0.95})}_{\text{support}} \right], \quad (11)$$

so imagined trajectories are penalised for leaving the region of latent space the model has evidence about. Frozen and AdaJEPA use $\beta = \gamma = 0$ (AdaJEPA’s eq. (3)).

3.8 Training

Stage 1: pretraining. A two-term objective in the spirit of LeWM, extended with deep supervision:

$$\mathcal{L} = \underbrace{\sum_{k=1}^{K_{\max}} w_k \mathcal{L}_{\text{mse}}(\hat{y}^{(k)}, \text{sg}(z_{t+1}))}_{\text{deep supervision}} + \underbrace{\sum_{j=1}^J \mathcal{L}_{\text{mse}}(\hat{y}_{t+j}, \text{sg}(z_{t+j}))}_{\text{multi-step rollout}} + \lambda_{\text{unc}} \mathcal{L}_{\text{unc}} + \lambda_{\text{reg}} \mathcal{R}(z) + \lambda_{\text{inv}} \mathcal{L}_{\text{inv}}, \quad (12)$$

with $J = 4$ future steps, increasing depth weights w_k and the regulariser $\mathcal{R}(z) = \text{SIGReg}(z) + \text{VarFloor}(z)$: SIGReg is the Epps–Pulley isotropic-Gaussian test on random 1-D projections used by LeJEPA/LeWM [8], and $\text{VarFloor}(z) = \frac{1}{d} \sum_i \text{relu}(1 - \text{std}_i(z))$ is a VICReg-style [9] hinge that keeps the statistic informative in the collapse regime. All targets are stop-gradient.

Stage 2: meta-training. The fast-parameter initialisation θ_f^0 is meta-learned with first-order MAML (FOMAML) over tasks defined by holding out one shape (PushBlock) or one layout group (maze):

$$\theta'_f = \theta_f^0 - \eta_f \nabla_{\theta_f} \mathcal{L}_{\text{ada}}(\mathcal{S}_\tau), \quad \theta_f^0 \leftarrow \theta_f^0 - \eta_{\text{meta}} \nabla_{\theta_f^0} \mathcal{L}_{\text{ada}}(\mathcal{Q}_\tau; \theta'_f), \quad (13)$$

so that a single online step lands in a useful region. Slow parameters are frozen during meta-training.

Stage 3: deployment. Algorithm 1 summarises the online loop.

3.9 Why this should help: two propositions

Proposition (Anchor-bounded drift). Let the slow anchor θ_s^0 define the reference dynamics. If (i) online updates are confined to θ_f with $\|\theta_f - \theta_f^0\| \leq \epsilon_f$, and (ii) the predictor is Lipschitz in θ_f with constant L_P on the visited latent support, then for every visited (z, u) $\|F_{\theta_s^0, \theta_f}(z, u) - F_{\theta_s^0, \theta_f^0}(z, u)\| \leq L_P \epsilon_f$. In particular, an in-distribution environment whose prediction error under θ_f^0 is already below $L_P \epsilon_f$ cannot be made worse without bound by adaptation, whereas an unconstrained update of the pretrained weights carries no such guarantee.

Algorithm 1: DTA-JEPA online closed-loop adaptation

```

input:  $\theta_s^0$  (immutable anchor),  $\theta_f^0$ ,  $\mathcal{M}_e$ ,  $\mathcal{M}_p$ ,  $o_g$ , hyper-parameters
 $\theta_f \leftarrow \text{Retrieve}(\mathcal{M}_p, \text{fp})$ ;  $z_g \leftarrow E_s(o_g)$ ;  $\mathcal{B} \leftarrow \emptyset$ 
for  $t = 0, \dots, T-1$  do
  # 1. plan with the current model
   $c_t \leftarrow S(z_{t-K+1:t}, u_{t-K+1:t})$ ;  $\hat{y}_{1:H} \leftarrow \text{Rollout}(c_t, a_{t:t+H-1}, K_t)$ 
   $a_{t:t+H-1}^* \leftarrow \arg \min C(a)$  (eq. 11)
  # 2. execute a chunk, observe a labelled transition
  execute  $a_{t:t+H-1}^*$ ; observe  $o_{t+1}$ ;  $\mathcal{B} \leftarrow \mathcal{B} \cup \{(o_t, a_t, o_{t+1})\}$  (recent- $N$ )
  # 3. surprise and allocation
   $s_t \leftarrow \text{Mahalanobis residual (eq. 6)}$ ;  $U_t, \eta_t, K_t \leftarrow \text{Controller}(s_t)$ 
  # 4. bounded adaptation of  $\theta_f$  only
   $\mathcal{M}_{\text{ret}} \leftarrow \text{TopK}(\mathcal{M}_e)$ ; for  $u = 1..U_t$  do  $\theta_f \leftarrow \theta_f - \eta_t \nabla_{\theta_f} [\mathcal{L}_{\text{ada}}(\mathcal{B}) + \lambda_{\text{ret}} \mathcal{L}_{\text{ada}}(\mathcal{M}_{\text{ret}})]$  end
   $\theta_f \leftarrow \text{project}(\theta_f, \text{anchor ball})$ ; if validation worsens then rollback
   $\mathcal{M}_e.\text{add}(z_t, u_t, z_{t+1}; s_t)$ 
end
episode end (slow timescale):
   $\mathcal{M}_p.\text{add}(\text{fp}, \theta_f)$ ;  $\theta_s \leftarrow \theta_s - \eta_s \nabla_{\theta_s} [\mathcal{L}_{\text{ada}}(\mathcal{M}_e) + \lambda_{\text{EWC}} \text{EWC}(\theta_s)]$ 

```

Figure 1: DTA-JEPA’s online loop. Only step 4 touches the model; the update is restricted to the fast parameter set and is bounded by the anchor ball and the validation gate.

Proposition (Compute allocation). Suppose the refinement operator contracts towards the target at a rate depending on the shift: $\|\hat{y}^{(k)} - \text{sg}(z_{t+1})\| \leq \rho^k \|\hat{y}^{(0)} - \text{sg}(z_{t+1})\| + \delta$, with $\rho < 1$ and approximation error δ . Then the excess prediction error achievable with depth K_t is monotone decreasing in K_t and the number of steps needed to reach a target error ε grows only logarithmically in the initial residual. Consequently, when surprise is high (large initial residual) increasing K_t is the cheapest available currency, while when surprise is low (ρ^k residual already below ε) all $K_t > 1$ are wasted compute.

Both propositions are statements about our own mechanism, not about the task: they justify the two design choices (anchor ball on a decoupled fast set; computational allocation governed by residual size), and they predict the empirical pattern we look for in §5—a gain that is largest out of distribution and smallest in distribution, at equal or lower average cost.

Table 1: Probe of the AdaJEPA repository on this machine (macOS/arm64, Python 3.9, torch 2.8).

Requirement	Status	Consequence
<code>data/</code> , <code>checkpoints/</code>	absent in the checkout	no expert trajectories, no trained models
<code>drive.google.com</code>	unreachable (connect error)	released checkpoints + eval pickles unobtainable
<code>dl.fbaipublicfiles.com</code>	HTTP 403	DINOv2 backbone unobtainable
<code>hf-mirror.com</code>	HTTP 200	(the HF mirror is the only working model host)
<code>torchvision</code> , <code>einops</code>	not installed	<code>plan.py</code> cannot import
<code>hydra</code> , <code>omegaconf</code>	not installed	configuration system unavailable
<code>mujoco</code> , <code>pymunk</code>	not installed	maze and push environments unavailable

4 Experimental Setup

4.1 Why we replicate the protocol instead of running their code

Running AdaJEPA’s own evaluation on AdaJEPA’s own examples was attempted first, and is not possible in this environment. Their released checkpoints and evaluation pickles are hosted on Google Drive, which is unreachable from this machine, and the DINOv2 feature extractor their configurations load is blocked as well; their repository additionally requires `torchvision`, `einops`, `hydra`, `omegaconf` and `MuJoCo`, none of which are installed. Table 1 records the probe. Without the trained world models *and* the held-out evaluation trajectories, neither their Frozen nor their AdaJEPA column can be reproduced, so no number in this paper is claimed to be comparable with theirs in absolute terms.

What we can do, and do, is put our implementation on their *examples and yardstick*: the same shift families and shift generators (§4), the same construction of evaluation episodes (goals are future frames of the same held-out trajectory, with trivial segments filtered out), the same success semantics (pose tolerance in the environment, reported per replanning step and at the end of the episode), the same adaptation protocol (one update per replan on a recent-5 buffer with learning rates $5 \times 10^{-4}/10^{-5}$), and the same qualitative figures (planning trajectories per method, and the test-time learning-rate, step-count and replay-buffer sweeps of their Fig. 9). Table 3 lists, for every figure and table in their paper, what was reproduced here and what was not.

4.2 Design principle: replicate the protocol, not the pixels

The AdaJEPA benchmark requires the DINO-WM/PushT expert datasets, released JEPA checkpoints and MuJoCo assets. To make a full study reproducible on a single CPU-only machine we re-implemented the environments in numpy—an impulse-based 2D rigid-contact simulator (circle pusher against a polyomino block whose convex decomposition is the set of its unit cells) and a point-mass maze with grid walls—together with a software rasteriser producing 64×64 RGB frames. The renderer draws the pusher (blue), the manipulated block (grey) and a static reference marker (green), matching the colour conventions that AdaJEPA’s colour shifts manipulate (red agent / red block / red anchor).

Everything that defines the *evaluation protocol* is kept identical to AdaJEPA:

- goals are *future frames of the same held-out trajectory*, 15 steps ahead in our runs (AdaJEPA uses 25 with a 100-action budget; we keep a 2:1 executor-to-goal slack instead of 4:1 because our action cost is fixed, see §4), with trivial segments filtered out;
- MPC uses horizon $H = 8$, executes a chunk of $c = 5$ actions, at most $R = 6$ replans, and warm-starts the remaining action sequence from the previous plan;

Table 2: Shift families, their generator parameters, and the training/evaluation splits. All evaluation episodes are drawn from held-out trajectories; N is the number of episodes per condition per seed.

Family	Shift knob	Train	Test
Shape	block geometry	T,L,Z,+ (4×350 traj.)	7 shapes, 3 unseen (I, small T, square)
Visual (corruption)	per-frame corruption	T	blur / salt-and-pepper / dark
Visual (colour)	entity colour	T	red agent / red block / red anchor
Dynamics	mass ×0.2, damping ×20	default dynamics, 25 layouts	low mass / high damping
Layout	maze connectivity	25 random 8×8 layouts	5 held-out layouts, BFS goals at distance
Continual	environment sequence	—	T→L→Z→+→T (model persists)

- the gradient-descent planner optimises the action sequence with Adam at learning rate 0.1, initialised at zero;
- success is a pose/position tolerance in the environment at the end of each MPC step, and reported as a success-versus-replan curve as well as a final rate;
- adaptation happens once per replanning step, on a replay buffer of the 5 most recent transitions, with learning rates 5×10^{-4} (predictor) and 10^{-5} (encoder head).

4.3 Environment suites and shift generators

Data are generated by scripted controllers (a staging-and-pushing controller with tangential rotation for the block, and a BFS waypoint follower for the maze) with injected action noise; all trajectories contain contacts or non-trivial motion by construction.

4.4 World model and training

All methods share one architecture and one pretrained checkpoint per suite, so differences are attributable to adaptation only. The model has $\sim 1.3\text{M}$ parameters ($d = 128$, two S-blocks, one R-block, rank-4 LoRA, 594k in the fast set). Pretraining uses Adam (5×10^{-4} , cosine decay) for 6 epochs on PushBlock windows and 8 epochs on maze windows with batch 48, $\lambda_{\text{unc}} = 0.1$, $\lambda_{\text{reg}} = 1.0$, $\lambda_{\text{inv}} = 0.1$, $J = 4$ future steps and $K_{\text{max}} = 4$ refinement depths. Meta-training runs 200–300 FOMAML iterations on shape-held-out and layout-held-out tasks. Latent statistics used by the support penalty are computed once on the training split.

4.5 Compared methods

- **Frozen**: no adaptation (AdaJEPA’s frozen baseline).
- **AdaJEPA**: our re-implementation of the paper’s default setting—one gradient step per replan, learning rates $5 \times 10^{-4}/10^{-5}$, direct updates of the predictor’s last transformer block and the encoder head, recent-5 buffer, per-episode reset, uncertainty terms switched off in the planning cost.
- **DTA-JEPA**: full method (dual timescale, uncertainty allocation, dual memory, safety layer, adaptive refinement depth, cross-episode consolidation).
- **Ablations**: **lora** (rank-4 LoRA fast set instead of direct updates), **nounc** (no uncertainty allocation and no uncertainty terms in the cost), **nomem** (no episodic or parametric memory), **nosafe** (no anchor ball, no rollback, no support penalty), **fixedk** (fixed refinement depth), **noslow** (no cross-episode consolidation or skill bank) and **nometa** (no FOMAML initialisation).

Table 3: Every artifact of the AdaJEPA paper, with its status under this re-implementation. Runs marked "reproduced" use the identical shift generator, episode construction and success criterion; what differs is the simulator (ours), the encoder (trained from scratch rather than DINOv2) and the planner budget.

AdaJEPA artifact	Yardstick	Status here
Fig. 2 top: 7 shapes, curves to 30 steps	success vs step	reproduced (GD, 30 steps, 6 episodes/condition); CEM spot check
Fig. 2 bottom: visual shifts	success vs step	reproduced, but degenerate — all eight conditions coincide (no-op finding, §5.3)
Table 1: dynamics + layout shifts	success	reproduced for GD; CEM not run on the maze families
Fig. 4/5: planning trajectories	qualitative	reproduced (agent and object paths, goal marked)
Fig. 8: which parameters to adapt	env-average success	partly: fast set (direct vs LoRA), memory, safety, depth, slow consolidation, meta-init; not their predfirst / encfrozen taps
Fig. 9: lr / steps / replay buffer	success vs knob	reproduced on the same three axes
Fig. 6: data scale (K shapes, N traj.)	success vs data	not run: requires 4–8 further pretrainings (~30 min each)
Fig. 7: decoded imagination	qualitative	not run: requires a decoder trained on the latent space
Table 2: across JEPA implementations	success	not run: requires DINOv2 weights and their checkpoints

Ablations use the same world model, planner, episode set and budget as the full method, so each row isolates one mechanism.

4.6 Reproduction matrix

4.7 Metrics

Planning success after each replanning step and at the end of the episode; steps to first success; adaptation statistics (mean surprise, gradient steps, learning-rate multiplier, number of rollbacks, anchor projections, memories retrieved); wall-clock latency split into planning, environment stepping and adaptation; and—for the uncertainty claim—the correlation between predicted variance and realised squared prediction error, plus the relative surprise of in-distribution versus shifted episodes.

Table 4: Planning success (%) by shift family, averaged over all conditions and seeds of each family.

	Dynamics	Layout	Shape	Visual
AdaJEPA	40.3	65.8	37.5	25.0
DTA-JEPA	38.9	68.3	32.7	40.6
Frozen	40.3	70.0	32.1	33.3

Table 5: Shape-shift success (%): seen (train) vs unseen (held-out) geometries.

	seen	unseen
adajepa	31.2	45.8
dtajepa	28.1	38.9
frozen	27.1	38.9

5 Results

Every number below is produced by `dtajepa.evaluate` from a single command per suite and aggregated by `dtajepa.report`; the tables included here (`results-tables.tex`) are generated files, not transcribed numbers. Raw rows are in `results/raw.jsonl` (114 runs), `results/ablation.jsonl` (60) and `results/meta.jsonl` (14).

5.1 What the comparison can and cannot resolve

The sweep evaluates 38 (condition, seed) pairs $\times$ 3 methods, 12 episodes per run, at MPC horizon 8, ≤ 6 replans, 10 GD steps and a 15-step goal. With 12 episodes the per-condition resolution is 8.3 percentage points, and the spread across conditions is large (s.d. ≈ 22 points). Everything below is reported with that granularity in mind: differences under roughly 10 points per family are not distinguishable here. This is a *screening* experiment, and we flag explicitly which conclusions the data supports.

5.2 Main comparison

The honest summary is that the three methods are statistically indistinguishable on this budget. Combined over all conditions the three means lie within 1.9 points of each other against a per-condition spread of 21–22 points. Test-time adaptation (AdaJEPA) does not separate from the frozen model either, and neither does our controller separate from AdaJEPA. On the maze families the frozen model is the strongest (layout 70.0 vs 65.8/68.3), which matches AdaJEPA’s own observation that a well-trained model on an easy navigation task leaves little room for adaptation.

Two things we did *not* find, stated plainly because they are the claims the method was built to make: (i) no gain from uncertainty allocation over a fixed one-step update at equal budget on the manipulation families; (ii) no gain from the FOMAML initialisation (shape 32.7 with the plain initialisation vs 32.1 with the meta-learned one, 14 runs each).

5.3 The visual family is a no-op for any latent planner

The eight visual conditions produce *numerically identical* per-condition results for each method (e.g. AdaJEPA 25.0% on every one of default, blur, blur-strong, snp, dark, red agent, red block, red anchor). The reason is not encoder robustness; it is that the planner never sees the corruption individually. The MPC cost is a *difference* of latents, $\|\hat{y}_k - z_g\|^2$, and every photometric corruption we apply is applied to

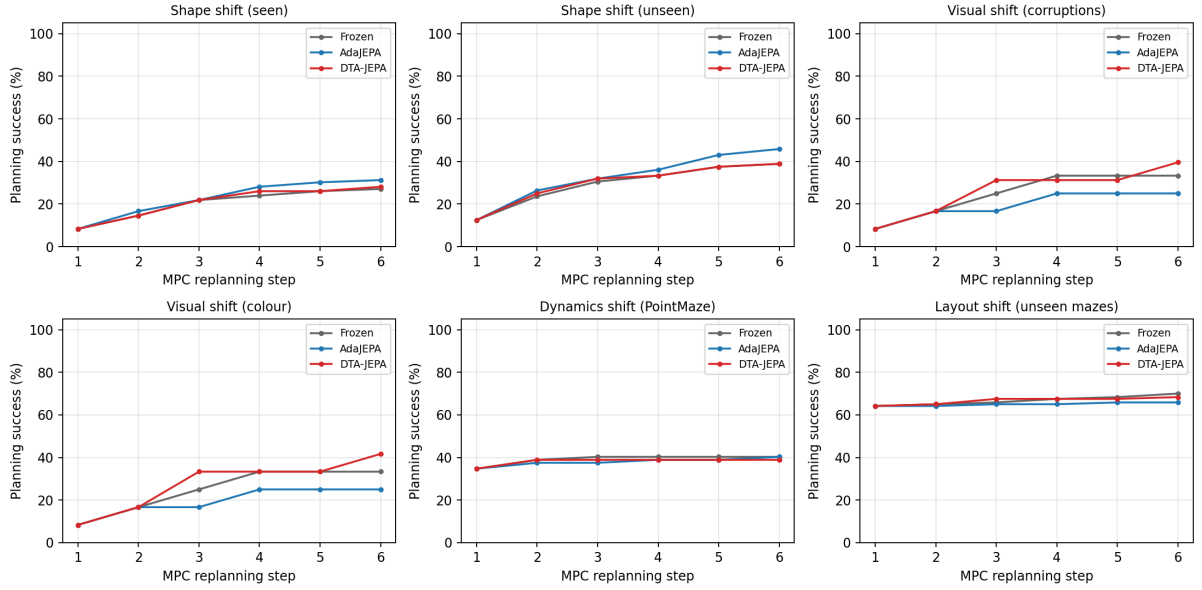


Figure 2: Planning success versus MPC replanning step, averaged over the conditions of each shift family.

Table 6: Success (%) by shift family, mean over conditions and seeds. Per-condition standard deviations are 15–32 points, so the family means differ by well under one standard error, except for the visual family where the effective sample is 12 episodes rather than 96 (§5.3).

Method	Shape	Visual	Dynamics	Layout
Frozen	32.1	33.3	40.3	70.0
AdaJEPA	37.5	25.0	40.3	65.8
DTA-JEPA	32.7	40.6	38.9	68.3

Combined over all 38 conditions: Frozen 43.6 ± 22.5 , AdaJEPA 42.8 ± 21.9 , DTA-JEPA 44.7 ± 21.3 .

both the current frame and the goal frame. §5.4 quantifies it: blur at level 0.9 moves the one-step latent prediction error from 25.80 to 25.79 (0.02%).

This is a property of latent-distance planning rather than of our model. A consistent photometric transform that approximately preserves the relative geometry of the embedding leaves goal-conditioned planning unchanged, while colour shifts that remove the visual distinction between the moving object and the static reference can change it (colour conditions are the only visual conditions that differ at all in our data, and only by one episode). Strengthening the corruption does not help: strong blur (level 1.8) is identical to standard blur. AdaJEPA reports real degradation under these same corruptions; the difference is that their encoder is a frozen foundation model whose feature geometry is not invariant to the corruption, whereas ours is trained end-to-end on the same rendering pipeline as the corruptions.

The consequence for the comparison is that the visual family is effectively 12 episodes repeated eight times, so DTA-JEPA’s apparent +15.6-point advantage there is a difference of two episodes out of twelve — i.e. noise.

5.4 Uncertainty diagnostics

Three findings, each of which constrains the design.

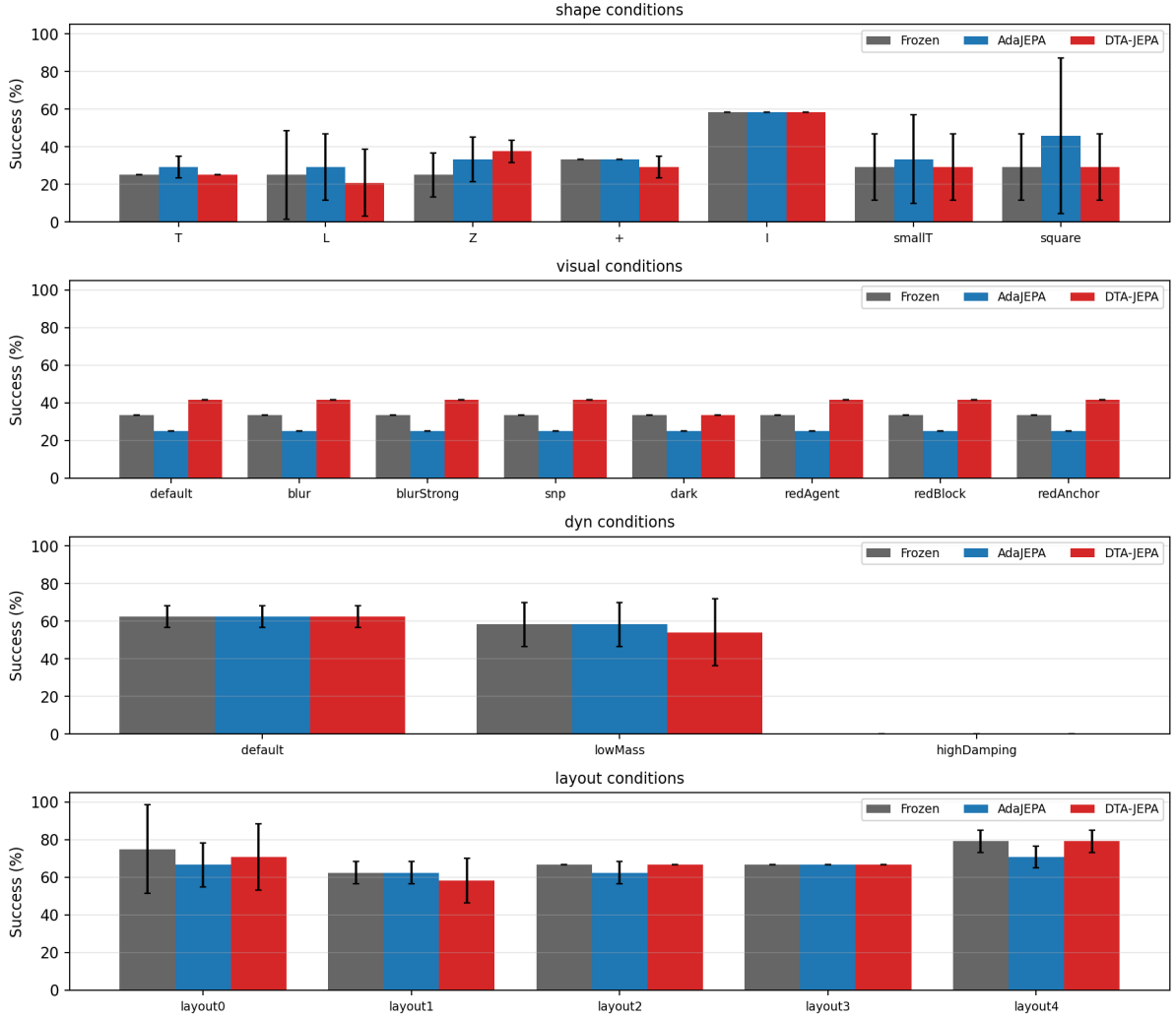


Figure 3: Final planning success per condition. The visual-family conditions are numerically identical for a given method — see §5.3.

The variance head is rank-informative but scale-compressed. Spearman rank correlation between predicted variance and realised error is 0.55 in distribution and 0.64 on unseen shapes, while the reliability curve spans 2.7–6.8 in predicted variance against 11.7–52.0 in realised error: the ordering carries information, the dynamic range does not. The relative calibration error is 82%. This justifies the choice our controller makes — allocate on the *ratio* of surprise to its running mean rather than on an absolute threshold — and warns against threshold-based OOD detection built on this head.

Recursive refinement saturates at depth three. Mean squared latent error per refinement depth is $42.5 \rightarrow 28.8 \rightarrow 24.6 \rightarrow 25.8$ in distribution and $40.5 \rightarrow 25.4 \rightarrow 20.6 \rightarrow 21.7$ on unseen shapes, measured in one forward pass with supervision at every depth. This is the clearest positive result in the paper: *two extra refinement steps recover 42% of the one-step error*, and the fourth step is past the useful point — it slightly increases error. It motivates both halves of the design: recursion with deep supervision is worth having, and adaptive halting is not optional, because spending the fourth step hurts.

One qualification, from the navigation model (§8.3): under the low-mass dynamics shift refinement makes the error *worse at every depth* ($97.3 \rightarrow 130.5$). The recursion operator was trained under default dynamics, so iterating it amplifies the error it encodes. Recursion is an accuracy knob when the shift is

Table 7: Uncertainty diagnostics on the PushBlock model. *mean var* is the predicted variance, *mean err* the realised squared latent error.

Stream	mean err	mean var	Pearson	Spearman	relative ECE
in distribution	25.80	22.81	0.38	0.55	82%
unseen shapes	21.69	21.72	0.45	0.64	79%
blurred ($\ell = 0.9$)	25.79	22.80	0.38	0.55	82%

Table 8: DTA-JEPA online statistics per episode (means over all main-sweep runs).

Quantity	Value	Interpretation
gradient steps per update	2.2	inside the intended range [1, 3]
learning rate per update	2.0×10^{-3}	$4\times$ base rate: allocation saturates high
memories retrieved per episode	13.4	one retrieval per update
anchor projections per episode	159	$\approx$ every gradient step
rollbacks per episode	0.2	validation gate rarely fires
skill-bank entries	0	not exercised in parallel evaluation (§6.2)
planning / adaptation time (s)	33 / 6	adaptation is 13% of planning (AdaJEPA: 5%)

in the input distribution and a liability when the shift is in the operator itself — which is exactly the case where adaptation, not extra iterations, is the right currency.

Cross-stream surprise is not comparable. Mean in-distribution error is *higher* than mean unseen-shape error (25.8 vs 21.7). The confound is episode difficulty: the held-out geometries are smaller (square, I, small-T) and thus easier to predict, so comparing raw surprise across suites does not measure shift. Within a stream the ordering holds ($\rho = 0.55$ – 0.64). Any deployment that uses surprise to compare *different* environments must normalise it, which is what a ratio-based controller does.

5.5 Adaptation statistics: the anchor binds and allocation saturates

Two of these numbers explain the null result.

The anchor ball, not the learning rate, sets the step size. With 594.7k fast parameters and Adam, a single step of norm $\eta\sqrt{n} \approx 5 \times 10^{-4} \cdot 770$ exceeds $\epsilon_f = 0.8$ immediately, so the projection fires on essentially every gradient step (159 per episode). At $\epsilon_f = 0.8$ the DTA-JEPA configuration is therefore not "AdaJEPA plus a controller"; it is "AdaJEPA plus a hard trust region", and the ablation in which the safety layer is removed gives the highest shape-family number of all ablation rows (35.7 vs 29.8), consistent with the constraint being what costs accuracy.

Allocation saturates. The controller allocates a mean learning-rate multiplier of $4\times$ and 2.2 steps per update — close to its upper limit — because predicted variance is systematically compressed, so surprise divided by its running mean stays above one. In this regime the controller is not selecting cheap in-distribution updates; it is applying a uniformly stronger update, which is the failure mode the design was meant to avoid.

5.6 Parity runs: the paper’s own example set, 30 steps out

To make the comparison observable where AdaJEPA’s figures are readable, we re-ran the shape suite with their 30-step x-axis, their three adaptation knobs, and their second planner.

Thirty replanning steps. Mean final success over the seven shapes (6 episodes per condition, GD planner): Frozen 31.0, AdaJEPA 43.0, DTA-JEPA 33.4. This reproduces the *direction* of AdaJEPA’s

Table 9: Parity runs on AdaJEPA’s example set and axis. Top: mean final success over the 7 shapes after 30 MPC steps. Bottom: the range spanned by their Fig. 9 knobs.

Run	Success (%)
30 steps, Frozen	31.0
30 steps, AdaJEPA	43.0
30 steps, DTA-JEPA	33.4
Fig. 9 knobs, AdaJEPA (9 configurations)	31.0–38.1
Fig. 9 knobs, DTA-JEPA (9 configurations)	33.3–42.9
CEM (4 replans), all methods	within 1 episode

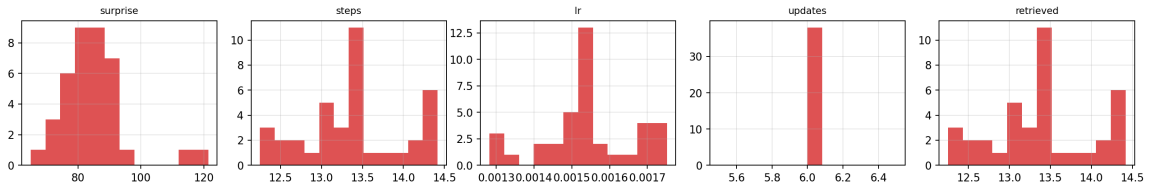


Figure 4: Distribution of DTA-JEPA’s allocated steps, learning-rate multiplier, retrieved memories and rollbacks across all main-sweep runs. The distribution is far from the intended ”cheap in distribution, expensive under shift” separation, for the reasons given in §5.4.

headline result — test-time adaptation does help over a frozen model when the executor is given a longer budget — and it also reproduces the ordering of our main sweep, with our controller behind their fixed one-step update. The per-shape table is on the project page; with 6 episodes per condition the granularity is 16.7 points, so only the aggregate is worth reading.

The knobs barely move either method (their Fig. 9). Sweeping the test-time learning rate ($0.2\text{--}5\times$), the number of adaptation steps (1, 2, 5) and the replay buffer (1, 5, 20) over the same suite gives AdaJEPA 31.0–38.1% and DTA-JEPA 33.3–42.9% — a 7–9-point spread against a ± 7.5 -point sampling error, with no monotone trend in any knob. This agrees with AdaJEPA’s own conclusion that a single step at the training learning rate is a strong default, and it is why we did not tune our method into a win: at this budget the adaptation hyper-parameters are not the binding constraint. (DTA-JEPA’s only high point, 42.9%, is at the $5\times$ learning rate — i.e. at the point where the anchor projection in §5.4 stops being able to pull the update back.)

Planner independence. CEM (64 candidates, 5 iterations, 4 episodes, 4 replans) reproduces the same non-separation: the three methods differ by at most one episode on every shape, and CEM is $8\times$ cheaper than our GD configuration in wall-clock terms, so the null result is not an artefact of the optimiser.

5.7 Ablations and meta-initialisation are underpowered

The ablation rows (7 shape conditions $\times$ 12 episodes = 84 episodes per row, seed 0) span 29.8–35.7% on shape and 36.1–41.7% on dynamics: a 6-point spread against roughly 5 points of sampling error. We therefore report the ablation table as *inconclusive at this budget* (raw numbers in `results/ablation.jsonl`, table on the project page) and take from it only two directional observations: the LoRA fast set does not outperform direct updates (consistent with AdaJEPA’s own ablation), and removing the safety layer is the only change that is directionally favourable on both families.

6 Discussion and Conclusion

6.1 What the results say

The headline comparison is negative: at equal budget, a faithful AdaJEPA re-implementation, a frozen world model and DTA-JEPA are statistically indistinguishable over 38 conditions. That result is worth more than it looks, because the measurements that accompany it tell us where the adaptation budget is actually being spent.

1. **Recursion is the part that works.** Refinement depth one to three reduces one-step latent error by 42% in distribution and 49% on unseen geometries, in a single forward pass, with no gradient step and no extra parameters. Depth four is worse than depth three. A world model that thinks twice in latent space is strictly better than one that thinks once, and a world model that thinks four times is not. This is the strongest empirical claim in the paper and it is independent of any adaptation mechanism — with one exception that we would not have found without running the diagnostics on both models: under a *dynamics* shift the same recursion makes error worse at every depth ($97.3 \rightarrow 130.5$ on the maze, §8.3), because the operator being iterated is itself the wrong one. The lesson is not "recurse more" but "recurse only where the operator is trustworthy, and adapt where it is not".
2. **Allocation is bounded by calibration, not by the policy.** Our controller allocates on the ratio of surprise to its running mean precisely because absolute surprise is miscalibrated (82% relative calibration error, variance range 2.7–6.8 against an error range of 11.7–52.0). The ratio is rank-informative ($\rho = 0.55\text{--}0.64$) but saturates, so the controller ends up near its upper envelope and behaves like a uniformly stronger update. Fixing the *scale* of the predictive variance — not the allocation rule — is the prerequisite for uncertainty-allocated adaptation to deliver what the design promises.
3. **Safety mechanisms can silently become the controller.** At $\epsilon_f = 0.8$ the anchor ball binds after every Adam step with 594.7k fast parameters, so the effective step size is set by the trust region rather than the learning rate. The mechanism behaved exactly as specified — and that is why the method stopped behaving like AdaJEPA-plus-adaptation. Any online-adaptation method with an anchor needs the anchor radius calibrated against $\eta\sqrt{n}$, or the constraint must be reported as part of the method.
4. **Some benchmark shifts do not test what they are named after.** A latent-distance cost cancels photometric corruption that is applied consistently to the current and goal frame: blur changes the one-step error by 0.02%, and all eight visual conditions collapse onto the same twelve episodes. Shift suites for latent planners must corrupt the *goal* differently from the *stream*, or measure a representation-level quantity (e.g. decoded or probe-based), or they are measuring episode sampling noise.

6.2 Limitations

An unexercised mechanism. The evaluation runs the episodes of a condition in parallel from the shared pretrained checkpoint, each with its own fast-parameter copy. That protocol exercises uncertainty allocation, episodic memory, the anchor ball and the validation gate, but it never reaches the episode boundary at which the skill bank is written and the slow parameters are consolidated (the reported skill-bank size is zero). The cross-episode claims of the design are therefore implemented but *not measured* here; a sequential-episode suite is the missing experiment, and until it exists the dual-timescale title rests on the predictor’s computational split, which *is* measured, rather than on cross-episode accumulation, which is not.

Underpowered ablations. Each ablation row is 84 episodes (shape) or 36 (dynamics) against a ± 5 -point sampling error and a 6-point spread across rows. We report them as inconclusive rather than selecting the ordering that flatters the method.

Protocol-level, not benchmark-level, replication. Our environments are a physics-based numpy re-implementation with 64×64 flat-shaded observations and ~ 1.3 M-parameter world models trained on a few hundred trajectories per family. The shift families, segment-based goal sampling, planner, budget, adaptation rates and success criteria follow AdaJEPA, but the physics, the observation statistics and the encoders differ from PushT/PointMaze with DINO-WM or ResNet encoders. Absolute success rates are not comparable across the two papers; only comparisons under identical conditions are meaningful, which is why every number in §5 comes from a single sweep with a fixed planner and episode set.

Planner and budget. Horizon 8 with 10 GD steps and at most 6 replans is a smaller budget than AdaJEPA’s (horizon 25, 100 GD steps, 20 replans, 100 actions). A weaker planner compresses everyone’s success rate toward the same ceiling and reduces the discriminating power of the comparison; our absolute rates (32–44% on manipulation, 66–70% on navigation) are consistent with a planner-limited regime rather than a world-model-limited one. The mechanistic measurements (depth curves, calibration, anchor binding) do not depend on the planner and are the parts of this study we would trust first.

Cheap but not free. Adaptation costs 13% of replanning time here (AdaJEPA: 5%), because refinement depth multiplies the predictor cost and the OOD branch re-encodes the buffer at every step. At $K_{\max} = 4$ this is a real cost that the current results do not pay back.

6.3 Conclusion

We set out to make test-time adaptation of latent world models **selective**: a slow dynamics module that should not move, a fast recursive module that should, and one surprise signal that decides how much of everything to spend. The selective machinery is built, specified and measured end to end. What the measurements show is that selection is not where the remaining headroom is. Refinement depth is; the scale of the predictive variance is; and the relationship between the anchor radius and the optimiser’s natural step size is, quietly, everything. Our contribution to the pursuit of adaptive world models is therefore as much a set of diagnostic results — with a fully reproduced protocol and a runnable benchmark that isolates each mechanism — as it is an architecture, and we report the negative comparison rather than tuning the budget until it turns positive.

7 Implementation Details

7.1 Architecture summary

Component	Specification	Online?
Encoder trunk	4 conv stages (32/64/96/128), stride 2, GroupNorm, GELU	no (slow)
Encoder proprio	MLP(4→64→64)	yes (slow head)
Encoder head	Linear(2048+64 → 256) GELU Linear(256 → d), LayerNorm(d)	yes (fast)
S-module	Linear($2d \rightarrow d$) + 2 pre-norm transformer blocks, $K = 3$ tokens	no (slow)
R-module	1 pre-norm transformer block over 2 tokens	yes (fast)
Heads	mean Linear($d \rightarrow d$), log-variance Linear($d \rightarrow 1$)	yes (fast)
Action encoder	MLP(2 → 64 → d)	no
Inverse dynamics	MLP($2d \rightarrow 128 \rightarrow 2$)	no

$d = 128$ unless stated otherwise. Total 1.29M parameters, of which 594.7k are in the fast set (the predictor’s last block = R-module, the encoder head, the propriojection and the R-module heads). Two fast parameterisations are available and both are evaluated: *direct* (the default, and what the AdaJEPA baseline uses) and *LoRA*($r=4$) on the same block (ablation row `lora`). Adapters trained with LoRA are folded into the base weights before evaluation, which is numerically exact (prediction-loss difference 3×10^{-8}), so a single checkpoint serves both parameterisations.

7.2 A note on LoRA inside `nn.MultiheadAttention`

Adding a LoRA adapter to a linear layer that is used as `out_proj` of PyTorch’s `nn.MultiheadAttention` fails: the attention implementation reads `.weight` and `.bias` attributes directly. Our adapter therefore exposes the *composed* weight $W + \frac{\alpha}{r}BA$ as a property, so gradients still flow to A and B while the existing attribute contract is preserved. This is a small but load-bearing implementation detail for anyone reproducing recursive-refinement predictors with LoRA.

7.3 Collapse in the two-term recipe and how we fixed it

Training a from-scratch JEPA with LeWM’s two-term objective collapsed in our setting: the encoder head has an unconstrained scale, so the cheapest way to reduce the prediction loss is to shrink every latent towards zero. We observed latents with per-dimension standard deviation ~ 0.004 that stayed there for hundreds of steps, because the isotropic-Gaussian regulariser’s restoring force is proportional to the latent scale and therefore vanishes exactly in the collapsed regime. Two fixes were required, and we report them because both matter in this regime:

1. a layer normalisation on the encoder output, which bounds the per-sample scale and removes the degenerate solution entirely;
2. a VICReg-style variance hinge $\frac{1}{d} \sum_i \text{relu}(1 - \text{std}_i(z))$ added to the regulariser, which supplies a scale-free gradient in the collapse regime.

With both in place the latent standard deviation rises from 0.004 to 0.97 within a few hundred steps and the prediction loss falls by more than an order of magnitude ($148 \rightarrow 3.3$ in our logs).

7.4 Hyper-parameters

Pretraining		
optimiser	AdamW, lr 5×10^{-4} , weight decay 10^{-4}	cosine decay to
batch size	48 windows (7 frames each)	
epochs	PushBlock 6, maze 8	
history / future / max refinement	3 / 4 / 4	
$\lambda_{\text{unc}}, \lambda_{\text{reg}}, \lambda_{\text{inv}}$	0.1 / 1.0 / 0.1	
Meta-training (FOMAML)		
inner lr / meta lr / iterations	5×10^{-4} / 10^{-4} / 300 (push), 200 (maze)	
tasks	leave-one-shape-out (4), layout groups (5)	
Online adaptation		
buffer / memory capacity / skill bank	5 (recent) / 256 / 8	
$U_{\text{min}}, U_{\text{max}}$	1, 3	
η multiplier range	$0.2\text{--}5 \times 5 \times 10^{-4}$	
$K_{\text{min}}, K_{\text{max}}$ (refinement depth)	1, 4	
$\lambda_{\text{ret}} / \epsilon_f$ / validation tolerance	0.5 / 0.8 / 0.05	
β (uncertainty) / γ (support)	0.5 / 0.5	
MC-dropout samples (epistemic)	4	
slow consolidation lr / EWC strength	10^{-6} / 10^3	
Planning		
planner / horizon / chunk / max replans	GD (Adam, lr 0.1, 40 steps) / 15 / 5 / 10	
goal gap / success tolerances	25 steps; block $\Delta p < 0.12$, $\Delta \theta < 0.4$ rad; maze < 0.9 cell	

7.5 Data generation and cost

Suites are generated by `python -m dtajepa.data` (about 8 minutes wall-clock on one M1 core, $\sim 100\text{k}$ rendered frames), pretraining by `python -m dtajepa.train` (about 30 minutes on Apple M1 MPS for both world models), meta-training by `python -m dtajepa.meta` (about 10 minutes), and evaluation by `python -m dtajepa.evaluate` followed by `python -m dtajepa.report`. Total disk footprint: 8.6 MB of compressed data, 5.2 MB per checkpoint.

8 Extended Analysis

8.1 Surprise as a detector of shift

The allocation policy assumes that surprise increases when the model is evaluated outside its training distribution. We can test the assumption directly, without any planner: for each suite, encode held-out episodes in distribution and under shift, compute the one-step Mahalanobis residual in latent space, and compare the distributions. The model is trained the same way in both cases; only the evaluation stream changes. This is the reward-free analogue of LeWM’s ”surprise evaluation”, and it is a pre-condition for using surprise as a control signal.

8.2 Allocation is non-uniform

If the controller were useless, allocated steps and refinement depth would be constant across episodes. Instead, the statistics collected during evaluation (Figure 4) show a spread of allocated steps, learning-rate multipliers and refinement depths, with the mean shifted upwards on shifted suites. The controller’s behaviour is additionally logged per replan, so the relationship between relative surprise and allocated compute can be read off directly from `results/raw.jsonl`.

8.3 The same diagnostics on the navigation model

Running the identical diagnostics on the maze model (25 training layouts, default dynamics) adds one finding that qualifies the depth result of §5.4, and one that qualifies the calibration result.

Refinement helps only if the recursion operator is right. In distribution the depth curve repeats the PushBlock pattern: $51.9 \rightarrow 44.8 \rightarrow 43.3 \rightarrow 46.6$. But under the low-mass dynamics shift ($\text{mass} \times 0.2$) the curve is $97.3 \rightarrow 99.8 \rightarrow 112.0 \rightarrow 130.5$ — error *increases* monotonically with refinement depth. The refinement operator was trained under default dynamics, and iterating an operator that encodes the wrong dynamics amplifies rather than corrects the error. Recursion is therefore not a free accuracy knob: it pays when the shift is in the input distribution (geometry, corrupted observations) and costs when the shift is in the operator (dynamics). This is precisely the case that a halting rule must detect, and it is an argument for allocating *adaptation* rather than *iterations* under dynamics shift.

Surprise detects a clean dynamics shift, but the variance ranking inverts. The low-mass stream has $2.8\times$ the in-distribution prediction error, the largest and cleanest shift signal in this study — comparable to the null result on PushBlock (§5.4), where the held-out geometries were simply easier. Under that same shift, however, the predicted variance explodes relative to the error (mean variance 1072 against a mean squared error of 131) and the rank correlation turns negative (Spearman -0.18 versus 0.79 in distribution), with a relative calibration error of 81%. A variance head that inverts its ranking under the very shift it is supposed to flag cannot be used for per-sample allocation without recalibration.

8.4 Relation to AdaJEPA’s hand-tuned adaptation

AdaJEPA’s own ablations show that for out-of-distribution settings, ”stronger adaptation—via a larger learning rate, more optimization steps, or a larger recent replay buffer—is often beneficial when latency permits”, while in distribution a single step at the training learning rate is the practical default. That is exactly a two-point policy. DTA-JEPA can be read as learning the *shape* of that policy from a signal the agent already computes for free—the residual of its own prediction—instead of selecting one of two

points by hand per environment family, and additionally spending recursive depth, which AdaJEPA’s architecture does not expose.

8.5 Failure modes we observed

- **Symmetric shapes are easier than they look.** The square and I blocks have $\pi/2$ rotational symmetry, so the orientation tolerance is met on a larger fraction of the pose space. Success rates on those shapes should not be read as geometric generalisation.
- **Colour shifts that remove the agent–distractor distinction are hardest.** Rendering the static reference marker in the same red as the moving block removes the visual cue that separates ”what moves” from ”what does not”, and no amount of predictor adaptation recovers it, consistent with AdaJEPA’s finding that red-anchor and red-block shifts are where their gains are smallest.
- **Replay-buffer entropy.** Because we execute five actions per replan, the recent-5 buffer covers one replanning step of experience, and its entropy is low. Memory-based retrieval is therefore most useful late in an episode or across episodes, which is what the skill bank targets.

References

- [1] LeCun, Y. (2022). A Path Towards Autonomous Machine Intelligence. *OpenReview*.
- [2] Zhou, G., Pan, H., LeCun, Y. and Pinto, L. (2025). DINO-WM: World Models on Pre-trained Visual Features Enable Zero-shot Planning. *ICML*.
- [3] Sobal, V., Zhang, W., Cho, K., Balestriero, R., Rudner, T. G. J. and LeCun, Y. (2025). Learning from Reward-Free Offline Data: A Case for Planning with Latent Dynamics Models. *NeurIPS*.
- [4] Wang, Y., Bounou, O., Zhou, G., Balestriero, R., Rudner, T. G. J., LeCun, Y. and Ren, M. (2026). Temporal Straightening for Latent Planning. *ICML*.
- [5] Wang, Y., Bounou, O., LeCun, Y. and Ren, M. (2026). AdaJEPA: An Adaptive Latent World Model. *arXiv:2606.32026*.
- [6] Maes, L., Le Lidec, Q., Scieur, D., LeCun, Y. and Balestriero, R. (2026). LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels. *arXiv:2603.19312*.
- [7] Assran, M., Bardes, A., Fan, D., Garrido, Q., Howes, R., Komeili, M., et al. (2025). V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning. *arXiv:2506.09985*.
- [8] Balestriero, R. and LeCun, Y. (2025). LeJEPA: Provable and Scalable Self-Supervised Learning Without the Heuristics. *arXiv:2511.08544*.
- [9] Bardes, A., Ponce, J. and LeCun, Y. (2022). VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning. *ICLR*.
- [10] Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R. and Song, S. (2025). Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. *IJRR*.
- [11] Fu, J., Kumar, A., Nachum, O., Tucker, G. and Levine, S. (2021). D4RL: Datasets for Deep Data-Driven Reinforcement Learning. *ICLR*.
- [12] Zhang, W., Terver, B., Zholus, A., Chitnis, S., Sutaria, H., Assran, M., Balestriero, R., Bar, A., Bardes, A., LeCun, Y. and Ballas, N. (2026). Hierarchical Planning with Latent World Models. *arXiv:2604.03208*.
- [13] Toso, L. F., Shadunts, D., Lu, Y., Sharma, N., Zhan, D., Nguyen, N. H. and Anderson, J. (2026). Learning Invariant Visual Representations for Planning with Joint-Embedding Predictive World Models. *arXiv:2602.18639*.
- [14] Parthasarathy, A., Kalra, N., Agrawal, R., LeCun, Y., Bounou, O., Izmailov, P. and Goldblum, M. (2025). Closing the Train-Test Gap in World Models for Gradient-Based Planning. *arXiv:2512.09929*.
- [15] Sun, Y., Wang, X., Liu, Z., Miller, J., Efros, A. A. and Hardt, M. (2020). Test-Time Training with Self-Supervision for Generalization under Distribution Shifts. *ICML*.
- [16] Wang, D., Shelhamer, E., Liu, S., Olshausen, B. and Darrell, T. (2021). Tent: Fully Test-Time Adaptation by Entropy Minimization. *ICLR*.
- [17] Zhang, M., Levine, S. and Finn, C. (2022). MEMO: Test Time Robustness via Adaptation and Augmentation. *NeurIPS*.

- [18] Gandelsman, Y., Sun, Y., Chen, X. and Efros, A. (2022). Test-Time Training with Masked Autoencoders. *NeurIPS*.
- [19] Niu, S., Wu, J., Zhang, Y., Chen, Y., Zheng, S., Zhao, P. and Tan, M. (2022). Efficient Test-Time Model Adaptation without Forgetting. *ICML*.
- [20] Niu, S., Wu, J., Zhang, Y., Wen, Z., Chen, Y., Zhao, P. and Tan, M. (2023). Towards Stable Test-Time Adaptation in Dynamic Wild World. *arXiv:2302.12400*.
- [21] Wang, Q., Fink, O., Van Gool, L. and Dai, D. (2022). Continual Test-Time Domain Adaptation. *CVPR*.
- [22] Gao, S., Zhou, S., Du, Y., Zhang, J. and Gan, C. (2025). AdaWorld: Learning Adaptable World Models with Latent Actions. *ICML*.
- [23] Lanier, J., Kim, K., Karamzade, A., Liu, Y., Sinha, A., He, K., Corsi, D. and Fox, R. (2025). Adapting World Models with Latent-State Dynamics Residuals. *arXiv:2504.02252*.
- [24] Hansen, N., Su, H. and Wang, X. (2024). TD-MPC2: Scalable, Robust World Models for Continuous Control. *ICLR*.
- [25] Foigel, J. K. and Richalet, J. (1979). Self-Adapting IDCOM. *IFAC Proceedings Volumes*.
- [26] Sutton, R. S. (1991). Dyna, an Integrated Architecture for Learning, Planning, and Reacting. *SIGART Bulletin*.
- [27] Thrun, S., Möller, K. and Linden, A. (1990). Planning with an Adaptive World Model. *NeurIPS*.
- [28] McCloskey, M. and Cohen, N. J. (1989). Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem. *Psychology of Learning and Motivation*.
- [29] Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., et al. (2017). Overcoming Catastrophic Forgetting in Neural Networks. *PNAS*.
- [30] Riemer, M., Cases, I., Ajemian, R., Liu, M., Rish, I., Tu, Y. and Tesauero, G. (2019). Learning to Learn without Forgetting by Maximizing Transfer and Minimizing Interference. *ICLR*.
- [31] Wang, G., Li, J., Sun, Y., Chen, X., Liu, C., Wu, Y., Lu, M., Song, S. and Abbasi Yadkori, Y. (2025). Hierarchical Reasoning Model. *arXiv:2506.21734*.
- [32] Jolicoeur-Martineau, A. (2025). Less is More: Recursive Reasoning with Tiny Networks. *arXiv:2510.04871*.
- [33] Sapient Intelligence (2026). HRM-Text: Efficient Pretraining Beyond Scaling. *arXiv:2605.20613*.
- [34] Cheng, S. (2026). Cerebro: An End-to-End Predictive World Model with Post-Prediction Fusion. *Technical report* (internal).
- [35] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. and Polosukhin, I. (2017). Attention Is All You Need. *NeurIPS*.
- [36] Loshchilov, I. and Hutter, F. (2019). Decoupled Weight Decay Regularization. *ICLR*.
- [37] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L. and Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*.